

Test Case ID	How I fixed the issue	Evidence (Key Words)
Example	<i>"At first, when I clicked the Add button, nothing happened. I realised I forgot to connect the command= in the Button to the function. I fixed this by adding command=add_task."</i>	command = function()
1	At first when I imported the blender file of the custom player character it did not load the blend file. I noticed that I had not allowed support for blender files for my game inside the Godot game project. Therefore I fixed this by changing the limitations of the project allowing blend files to be used as mesh instances.	(Nil)
2	At first when I imported a preset Godot outline shader, it only provided a inverted outline render rather than an outline. I noticed that the game inspector already had a preset stencil modifier for shaders allowing the same effect of an outline. I therefore did the following to fix this issue.	(Nil)
10	At first when testing the outline of the player characters, I had noticed that whenever another client had joined all joined clients would have the same coloured outline. I noticed that this was due to the fact that the stencil material was all the same and not unique for all character player nodes. Therefore I fixed this by ignoring the fact and decided to just appeal the colour recognition through the body mesh colours and will decide to re format the outline colour in another test case.	var outlinec := Color.WHITE @rpc("authority", "call_local") func set_color(new_color: Color, bodyc, eyec) -> void: for obj in body: var mesh: MeshInstance3D = obj var material: StandardMaterial3D = mesh.material_override var new_mat: StandardMaterial3D = material.duplicate() new_mat.albedo_color = bodyc new_mat.stencil_color = new_color mesh.material_override = new_mat if new_color == Color.GREEN: print("Client 2") for obj in eyes: var mesh: MeshInstance3D = obj var material: StandardMaterial3D = mesh.material_override var new_mat: StandardMaterial3D = material.duplicate() new_mat.albedo_color = eyec new_mat.stencil_color = new_color mesh.material_override = new_mat for obj in Obody: var mesh: MeshInstance3D = obj var material: StandardMaterial3D = mesh.material_override var new_mat: StandardMaterial3D = material.duplicate() new_mat.stencil_color = new_color mesh.material_override = new_mat for obj in Oeyes: var mesh: MeshInstance3D = obj var material: StandardMaterial3D = mesh.material_override var new_mat: StandardMaterial3D = material.duplicate() new_mat.stencil_color = new_color mesh.material_override = new_mat

18	At first when I tested the boundaries of the 3rd Map, I noticed that the collisions with the Bridge seems to catch the mesh collision of the player. This in turn caused the player to become stuck in teh gaps of the bridge. Therefore I noticed that the spacings between the collisions of the bridge and each board were unnecessary. I fixed this by only using one rectangular collision shape for each bridge which would decrease complexity and allows the player prevention from falling into unnecessary gaps in the bridge.	(Nil)
20	At first when I tested the first instance of a multiplayer system other than local hosting enet multiplayer, the client did not connect to the host client. This was mainly becuase the networking system was outdated causing this means to not work for what I had intended with their practice servers. Therefore I changed my method to a different networking system that was more up to date.	(Nil)
21	At first when I tested if the second window could connect to the host client over a network, it was unable to make a secure connection that was visible on both sides. I had noticed that this was mainly because I did not correctly assign the connection with both interfaces. Therefore I fixed this by opting for another networking system intead to allow easier multiplayer connection	(Nil)
22	At first when I tested the hosting system of the nodetunnel network, I noticed that the Host system was unable to connect to a viable network. I had noticed that this too did not have viable public testing servers that I could use for multiplayer connectivity. Therefore in order to salvage my work, I changed my networking system to a different networking system (Photon Engine)	(Nil)
25	At first when I tested the networking system, I noticed that players were unable to join over a network. This was mainly because my code was oreinted based on a local multiplayer strucutre. Therefore in order to fix this, instead of rewriting my code from the beginning I chose to choose another networking plugin that allowed for mutliplayer gameplay over a much more recent and prefered networking system. Therefore preventing me from having to rewrite my entire programme.	(Nil)

29

At first when testing mesh colour of the player upon joining the server, the player was unable to change its colour unique to its own client reciprocated upon other clients. This was mainly due to the fact that the mesh instances material was being shared upon the other player character bodies upon instantiation. In order to fix this problem, I had decided to temporarily change the mesh colour of the players to be set to the colours of the priority of the players (cyan, green, orange or red). Therefore by changing this allocation, I was able to ensure that the mesh colour did not depend on randomness since it was hard to reciprocate on other clients.

```
func colour_change(bodyc, eyec, outlinec):
if name == str(player_list.keys()[0]):
bodyc = Color.CYAN
elif name == str(player_list.keys()[1]):
bodyc = Color.GREEN
elif name == str(player_list.keys()[2]):
bodyc = Color.ORANGE
elif name == str(player_list.keys()[3]):
bodyc = Color.RED

GameManager.players = player_list.keys()

#outlinec = Color(randf_range(0, 1), randf_range(0, 1), randf_range(0, 1))
player_color = outlinec
body_color = bodyc
eye_color = eyec

set_color.rpc(outlinec, bodyc, eyec)

@rpc("authority", "call_local")
func set_color(new_color: Color, bodyc, eyec) -> void:
for obj in body:
var mesh: MeshInstance3D = obj
var material: StandardMaterial3D = mesh.material_override
var new_mat: StandardMaterial3D = material.duplicate()
new_mat.albedo_color = bodyc

new_mat.stencil_color = new_color
mesh.material_override = new_mat

if new_color == Color.GREEN:
print("Client 2")

for obj in eyes:
var mesh: MeshInstance3D = obj
var material: StandardMaterial3D = mesh.material_override
var new_mat: StandardMaterial3D = material.duplicate()
new_mat.albedo_color = eyec
new_mat.stencil_color = new_color
mesh.material_override = new_mat

for obj in Obody:
var mesh: MeshInstance3D = obj
var material: StandardMaterial3D = mesh.material_override
var new_mat: StandardMaterial3D = material.duplicate()
new_mat.stencil_color = new_color
mesh.material_override = new_mat

for obj in Oeyes:
var mesh: MeshInstance3D = obj
var material: StandardMaterial3D = mesh.material_override
var new_mat: StandardMaterial3D = material.duplicate()
new_mat.stencil_color = new_color
mesh.material_override = new_mat
```

30

At first when testing the shader for the xray of the players, it was unable to match the authority and reciprocate on all clients. This was mainly due to the fact that this also shared the same colour between instantiated character player bodies. Therefore in order to fix this I temporarily made this shader a set colour of white to ensure that no colour modifications were required making it easier to reciprocate on all clients while remaining visual insight on where the players are located behind objects in front of the camera.

```
func colour_change(bodyc, eyec, outlinec):
if name == str(player_list.keys())[0]:
bodyc = Color.CYAN
elif name == str(player_list.keys())[1]:
bodyc = Color.GREEN
elif name == str(player_list.keys())[2]:
bodyc = Color.ORANGE
elif name == str(player_list.keys())[3]:
bodyc = Color.RED

GameManager.players = player_list.keys()

#outlinec = Color(randf_range(0, 1), randf_range(0, 1), randf_range(0, 1))
player_color = outlinec
body_color = bodyc
eye_color = eyec

set_color.rpc(outlinec, bodyc, eyec)

@rpc("authority", "call_local")
func set_color(new_color: Color, bodyc, eyec) -> void:
for obj in body:
var mesh: MeshInstance3D = obj
var material: StandardMaterial3D = mesh.material_override
var new_mat: StandardMaterial3D = material.duplicate()
new_mat.albedo_color = bodyc

new_mat.stencil_color = new_color
mesh.material_override = new_mat

if new_color == Color.GREEN:
print("Client 2")

for obj in eyes:
var mesh: MeshInstance3D = obj
var material: StandardMaterial3D = mesh.material_override
var new_mat: StandardMaterial3D = material.duplicate()
new_mat.albedo_color = eyec
new_mat.stencil_color = new_color
mesh.material_override = new_mat

for obj in Obody:
var mesh: MeshInstance3D = obj
var material: StandardMaterial3D = mesh.material_override
var new_mat: StandardMaterial3D = material.duplicate()
new_mat.stencil_color = new_color
mesh.material_override = new_mat

for obj in Oeyes:
var mesh: MeshInstance3D = obj
var material: StandardMaterial3D = mesh.material_override
var new_mat: StandardMaterial3D = material.duplicate()
new_mat.stencil_color = new_color
mesh.material_override = new_mat
```

31

At first when also testing the outline
shader, I had occured to the same problem
as the xray shader regarding priority.
Therefore I had fixed this issue with the
same principle as followed.

```
func colour_change(bodyc, eyec, outlinec):
if name == str(player_list.keys())[0]:
bodyc = Color.CYAN
elif name == str(player_list.keys())[1]:
bodyc = Color.GREEN
elif name == str(player_list.keys())[2]:
bodyc = Color.ORANGE
elif name == str(player_list.keys())[3]:
bodyc = Color.RED

GameManager.players = player_list.keys()

#outlinec = Color(randf_range(0, 1), randf_range(0, 1), randf_range(0, 1))
player_color = outlinec
body_color = bodyc
eye_color = eyec

set_color.rpc(outlinec, bodyc, eyec)

@rpc("authority", "call_local")
func set_color(new_color: Color, bodyc, eyec) -> void:
for obj in body:
var mesh: MeshInstance3D = obj
var material: StandardMaterial3D = mesh.material_override
var new_mat: StandardMaterial3D = material.duplicate()
new_mat.albedo_color = bodyc

new_mat.stencil_color = new_color
mesh.material_override = new_mat

if new_color == Color.GREEN:
print("Client 2")

for obj in eyes:
var mesh: MeshInstance3D = obj
var material: StandardMaterial3D = mesh.material_override
var new_mat: StandardMaterial3D = material.duplicate()
new_mat.albedo_color = eyec
new_mat.stencil_color = new_color
mesh.material_override = new_mat

for obj in Obody:
var mesh: MeshInstance3D = obj
var material: StandardMaterial3D = mesh.material_override
var new_mat: StandardMaterial3D = material.duplicate()
new_mat.stencil_color = new_color
mesh.material_override = new_mat

for obj in Oeyes:
var mesh: MeshInstance3D = obj
var material: StandardMaterial3D = mesh.material_override
var new_mat: StandardMaterial3D = material.duplicate()
new_mat.stencil_color = new_color
mesh.material_override = new_mat
```

33	At first when testing the authority colour changing regarding the displayed mice on all clients, the mouse cursor did not correctly adhere to the following authorities that had joined the server. I had noticed that this was mainly because I had changed the directory that the mice would refer to when taking authority. Therefore I fixed this by changing the authority relative to the new directory change.	<pre>#add_player_rpc.rpc(peer_id) var player = PLAYER.instantiate() player.name = str(peer_id) get_tree().current_scene.get_node("Players").add_child(player) #get_tree().current_scene.get_node("Players").add_child(player) var mouse = MOUSE.instantiate() mouse.name = str(peer_id) get_tree().current_scene.get_node("CanvasLayer/PlayerCursors").add_child(mouse) #world.player_colour.emit()</pre>
36	At first when testing the card vanity change upon player mouse interaction, the control node had failed to display its card outline of a white thick line. This was mainly due to the fact that the card mouse interaction was set to the bottom layer and was overridden by the vanity outline interference. Therefore I had fixed this by ensuring the mouse interaction was valid behind the card interaction.	(Nil)
38	At first when testing the player spawning randomisation function, I noticed that the hosting player had not transported to the new global position. This was mainly due to the fact the annotation being called wrong. I had fixed this through the falling to call on all registered clients including the host authority/	<pre>@rpc("reliable") @rpc("any_peer", "reliable")</pre>